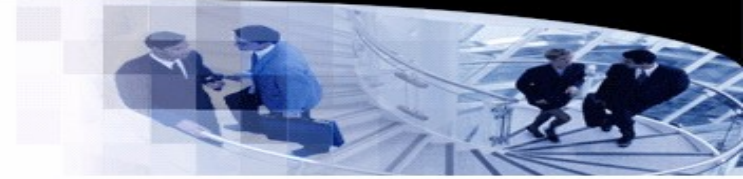




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

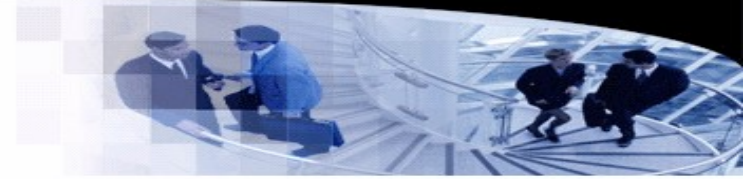
Lecture 3. Functional Programming

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Function Programming



- **Programming paradigm**

- Programs are constructed by applying and composing functions.
- Functions map values to other values.
- Pure functions have no side effects.

- **First-class functions**

- Functions can be bound to names, passed as arguments, and returned from other functions.

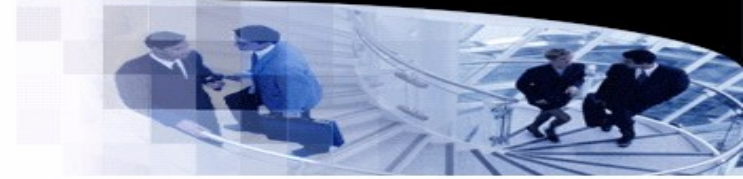
- **Pros**

- Readability, parallelism, compositionality, and declarative style.

- **Cons**

- Learning curve, performance, and debugging.

Recursion

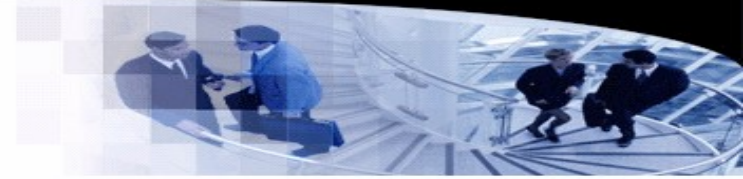


- Recursive function

- Use “let rec” instead of “let” to bind a recursive function.

```
# let rec fact x =  
    if x = 0 then 1  
    else x * fact (x - 1);;  
val fact : int -> int = <fun>  
# fact 4  
    ;;  
- : int = 24  
#
```

Recursion



- Recursive function

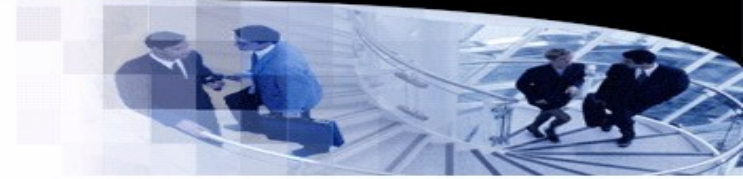
- Use “let rec” instead of “let” to bind a recursive function.

Keyword.

```
# let rec fact x =  
    if x = 0 then 1  
    else x * fact (x - 1);;  
val fact : int -> int = <fun>  
# fact 4  
;;  
- : int = 24  
#
```

If expression.

Recursion



- Tail recursion

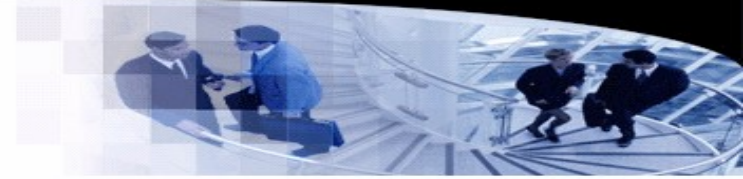
- Functional programs suffer from performance challenges caused by stack overhead.

```
unit -> 'a
let rec loop () = print_endline "Hello"; loop ();;

let _ = loop ();;
```

- Running the program does not lead to stack overflow. Why?
- Compilers can transform some recursions (e.g. tail recursions) into iterations.

Recursion



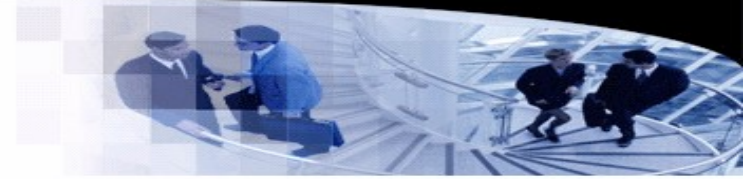
- Tail recursion

- A function is denoted as a tail-recursive function, if the recursive call happens at the end.

```
# let rec fact x =  
    if x = 0 then 1 else x * (fact [@tailcall]) (x - 1);;  
Warning 51 [wrong-tailcall-expectation]: expected tailcall  
  
val fact : int -> int = <fun>  
#
```

- fact is NOT tail-recursive, as * is the last call.
- [@tailcall] can be used to check tail calls.

Lists



- Definition

- A list is an ordered sequence of elements with the **same** type.

```
# [];;  
- : 'a list = []  
# [1; 2; 3];;  
- : int list = [1; 2; 3]  
# [1.0; 2.0; 3.0];;  
- : float list = [1.; 2.; 3.]  
# [[1]; [2;3]];;  
- : int list list = [[1]; [2; 3]]  
# [[1]; 2; 3];;  
Error: This expression has type int but an expression was expected of type  
      int list  
# █
```

Lists

• Definition

- A list is an ordered sequence of elements with the same type.

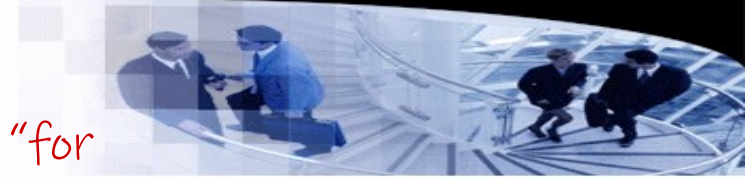
The empty list.

Type variable 'a' means "for any type 'a'".

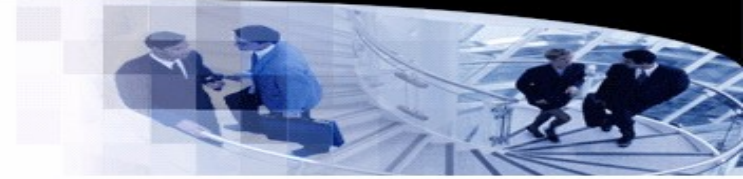
List of integers.

```
# [];;  
- : 'a list = []  
# [1; 2; 3];;  
- : int list = [1; 2; 3]  
# [1.0; 2.0; 3.0];;  
- : float list = [1.; 2.; 3.]  
# [[1]; [2; 3]];;  
- : int list list = [[1]; [2; 3]]  
# [[1]; 2; 3];;  
Error: This expression has type int but an expression was expected of type  
      int list  
#
```

A list which
contains a sequence
of lists.



Lists

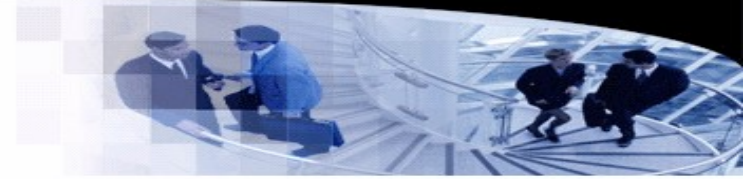


- Built-in operators

- `::` adds one element in the front of a list.
- `@` combines two lists.

```
# 1 :: [2; 3];;  
- : int list = [1; 2; 3]  
# [1] @ [2; 3];;  
- : int list = [1; 2; 3]  
# [1] :: [2; 3];;  
Error: This expression has type int but an expression was expected of type  
        int list  
# █
```

Lists



- Pattern matching

- A typical pattern matching structure is “match e with pattern-matching”.

```
# let foo x =  
  match x with  
    | 1 -> 2  
    | 2 ->      3  
    | _ -> 0;;  
val foo : int -> int = <fun>  
# foo 1;;  
- : int = 2  
# foo 10;;  
- : int = 0  
#
```

Lists

- Pattern matching

- A typical pattern matching structure is “match e with pattern-matching”.

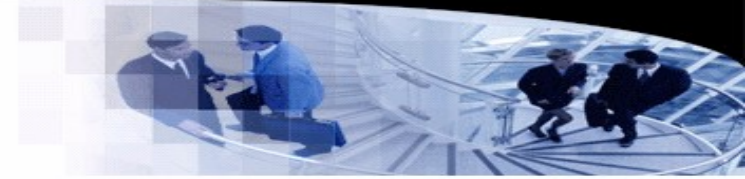
```
# let foo x =  
  match x with  
  | 1 -> 2  
  | 2 -> 3  
  | _ -> 0;;  
val foo : int -> int = <fun>  
# foo 1;;  
- : int = 2  
# foo 10;;  
- : int = 0  
#
```

Keyword.

Pattern.

Result for the
matched pattern.

Default pattern.



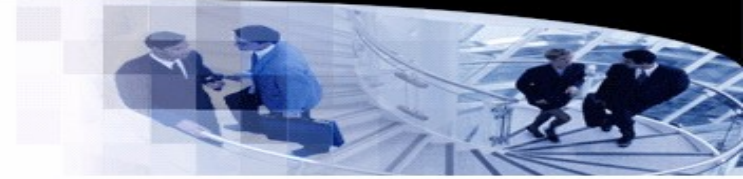
Lists

- Pattern matching

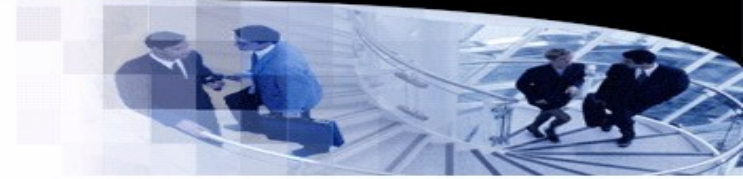
- Find the maximum element in a list.

```
'a list -> 'a
let rec list_max l =
  match l with
  | [] -> failwith "Empty list."
  | [x] -> x
  | h :: t -> max h (list_max t);;
```

```
# list_max [30; 12; 11; 45];;
- : int = 45
#
```



Lists



- Pattern matching

- Calculate the length of a given list.

```
'a list -> int
let rec length l =
  match l with
  | [] -> 0
  | _ :: t -> 1 + length t;;
```

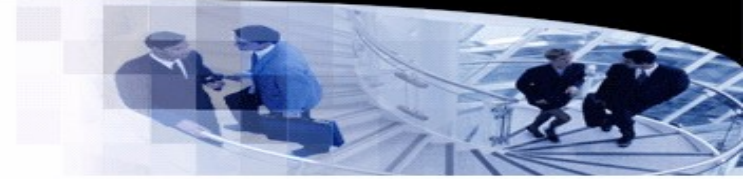
- However, it is NOT tail-recursive. Why?

Lists

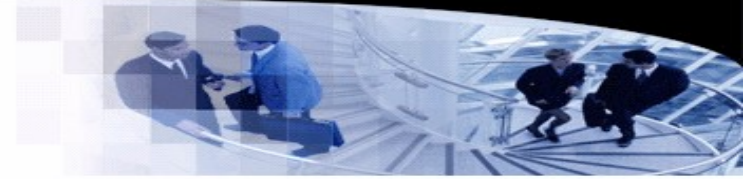
- Pattern matching

- Calculate the length via tail recursion.

```
int -> 'a list -> int
let rec length sum l =
  match l with
  | [] -> sum
  | _ :: t -> length (sum + 1) t;;
```



Lists

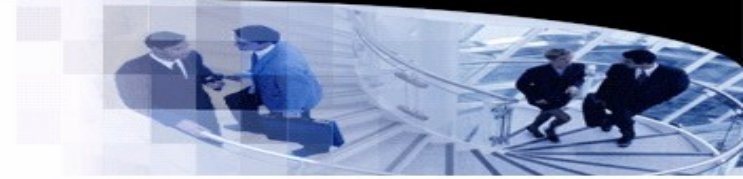


- Pattern matching

- Calculate the length via tail recursion. The wrapped version.

```
'a list -> int
let length l =
  let rec len sum l =
    match l with
    | [] -> sum
    | _ :: t -> len (sum + 1) t
  in len 0 l;;
```

Lists



- Higher order functions

- Map a list to another list, where each element is transformed by a given rule (function).

```
('a -> 'b) -> 'a list -> 'b list
let rec map f l =
  match l with
  | [] -> []
  | h :: t -> f h :: map f t;;
```

```
# map (fun x -> x + 1) [1; 2; 3];;
- : int list = [2; 3; 4]
# map (fun x -> length x) [[1]; [1; 2]; [1; 2; 3]];
- : int list = [1; 2; 3]
#
```


Lists

- Higher order functions

A higher order function that takes as argument a function.

- Map a list to another list, where each element is transformed by a given rule (function).

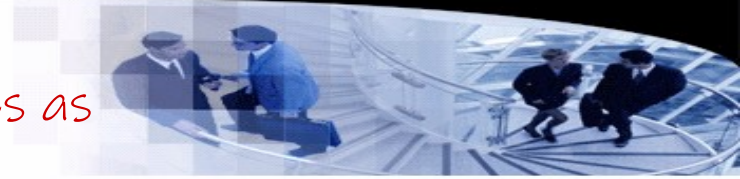
```
('a -> 'b) -> 'a list -> 'b list
let rec map f l =
  match l with
  | [] -> []
  | h :: t -> f h :: map f t;;
```

Transform an element by applying the function.

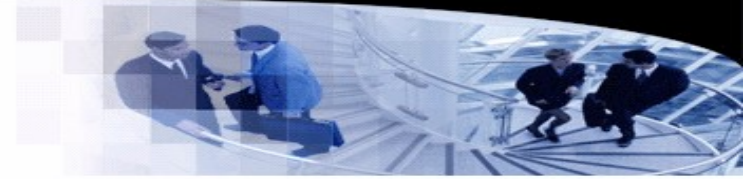
A function in the form of lambda term.

Calculate the length of a given list.

```
# map (fun x -> x + 1) [1; 2; 3];;
- : int list = [2; 3; 4]
# map (fun x -> length x) [[1]; [1; 2]; [1; 2; 3]];
- : int list = [1; 2; 3]
#
```



Lists



- Higher order functions

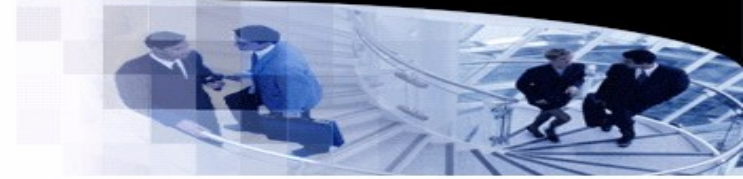
- Some standard list functions.
- Map two lists to another list:

```
# List.map2 ( + ) [1; 2; 3] [4; 5; 6];;  
- : int list = [5; 7; 9]  
# █
```

- Iterate a given list and do something:

```
# List.iter print_int [1; 2; 3];;  
123- : unit = ()  
# █
```

Lists



- Higher order functions

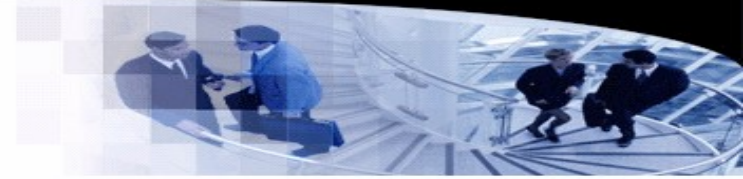
- Check whether an element is a member of a list:

```
# List.mem 2 [1; 2; 3];;  
- : bool = true  
# █
```

- Filter a list by a given predicate:

```
# List.filter (fun x -> x mod 2 = 0) [1; 2; 3; 4; 5; 6];;  
- : int list = [2; 4; 6]  
# █
```

Lists



- Higher order functions

- Folding: traverse a list and apply a given accumulator.

```
('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
let rec fold_left (f : 'a -> 'b -> 'a) (acc : 'a) (l : 'b list): 'a =
  match l with
  | [] -> acc
  | h :: t -> fold_left f (f acc h) t;;
```

```
# fold_left (fun acc x -> acc + x) 0 [1; 2; 3];;
- : int = 6
# fold_left (fun acc x -> acc * x) 1 [2; 3; 4];;
- : int = 24
#
```

Lists

A given accumulator.

Type annotation.

Base value.

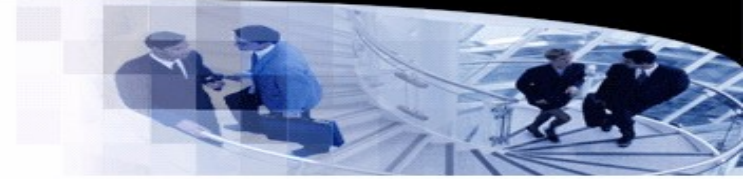
A list to traverse.

- Higher order functions

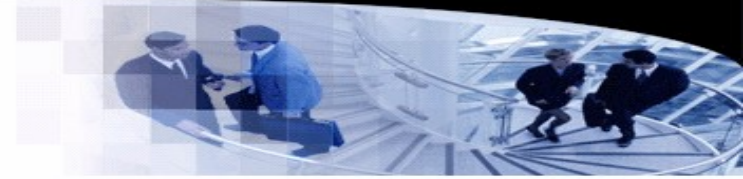
- Folding: traverse a list and apply a given accumulator.

```
('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
let rec fold_left (f : 'a -> 'b -> 'a) (acc : 'a) (l : 'b list): 'a =
  match l with
  | [] -> acc
  | h :: t -> fold_left f (f acc h) t;;
```

```
# fold_left (fun acc x -> acc + x) 0 [1; 2; 3];;
- : int = 6
# fold_left (fun acc x -> acc * x) 1 [2; 3; 4];;
- : int = 24
#
```



Lists



- Higher order functions

- Folding: the List module offers two folding functions, i.e., `fold_left` and `fold_right`.

```
# List.fold_left ( + ) 0 [1; 2; 3];;  
- : int = 6  
# List.fold_left max min_int [2; 4; 6; 0; 1];;  
- : int = 6  
#
```